Task Tracker, Line by Line

React, ASP.NET Core and SQL Server in Docker and Kubernetes

The dotnet-sqlserver task tracker explained file by file: the minimal API and EF Core, the React page, the containers and manifests, end-to-end tests against SQL Server, and the CI that runs them on every push.

Part	What it covers
0. The big picture	The pieces, the API, the layout
1. The API	Entity and rules, EF Core, minimal API endpoints, startup
2. The front end	React, a typed API client, nginx, client tests
3. Containers and Kubernetes	Dockerfiles, Compose, SQL Server in a StatefulSet, probes
4. Tests	xUnit v3, WebApplicationFactory, a database per test
5. Keeping it working	NuGet lock files, a vulnerable transitive package, CI with SQL Server

Every code listing shows the real file and its real line numbers. The notes under a listing refer to those numbers, so you can read the book with the project open beside it.

Contents

The big picture 4

0.1 The pieces 5

0.2 The API 5

0.3 The layout 6

The API 7

1.1 The project file 8

1.2 TaskItem.cs: the entity and its rules 8

1.3 AppDbContext.cs 9

1.4 Program.cs: setup 9

1.5 Program.cs: endpoints 10

1.6 Program.cs: starting up 11

1.7 Interview questions 12

The front end 13

2.1 Where the page comes from 14

2.2 The API client 14

2.3 The component 15

2.4 Development and production 17

2.5 Testing the client without a browser 18

2.6 Interview questions 19

Containers and Kubernetes 20

3.1 The API image 21

3.2 Docker Compose 21

3.3 Kubernetes 22

3.4 Interview questions 24

Tests 25

4.1 The test project 26

4.2 Unit tests: the title rule 27

4.3 A real app with a throwaway database 27

4.4 Running them 30

4.5 Interview questions 30

Keeping it working 31

5.1 NuGet lock files 32

5.2 A vulnerable package nobody chose 32

5.3 Semantic Versioning 33

5.4 The workflow 33

5.5 Dependabot 34

5.6 Interview questions 35

5.7 Exercise 35

Part 0

The big picture

The same task tracker as the Node version, with the backend in C#: an ASP.NET Core minimal API using Entity Framework Core and SQL Server. Same React page, same REST contract, same containers and manifests.

0.1 The pieces

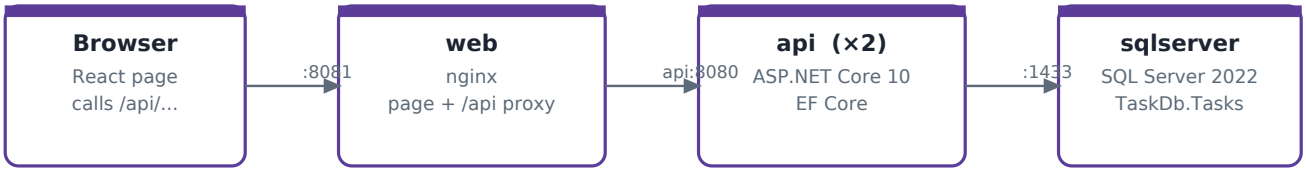


Figure 0.1. The browser asks nginx; nginx forwards /api to the API; EF Core talks to SQL Server.

Piece	Built with	Job
web	React 19, Vite, nginx	Serves the page; forwards `/api/` to the API
api	.NET 10, ASP.NET Core minimal APIs, EF Core 10	REST endpoints, validation, data access
sqlserver	SQL Server 2022 (Linux container)	Stores tasks in a volume

0.2 The API

Method and path	Does	Answers
GET /api/tasks	List tasks, oldest first	200 + array
POST /api/tasks	Create from `{ title }`	201 + Location + task; 400
PATCH /api/tasks/{id:int}	Change `title` and/or `done`	200 + task; 400; 404
DELETE /api/tasks/{id:int}	Delete	204; 404
GET /api/health/live	Is the process up?	200
GET /api/health/ready	Can it reach the database?	200 or 503
GET /api/info	Which pod answered	200 + `{ hostname }`

The routes, fields and status codes match the Node version, which is why the two can share a front end. One small difference: a non-numeric id like `/api/tasks/abc` is a 404 here (the route doesn't match) and a 400 in Node (the handler rejects it).

0.3 The layout

Path	What's there
`api/`	Program.cs (all endpoints and startup), TaskItem.cs (entity, requests, rules), AppDbContext.cs
`tests/TaskApi.Tests/`	xUnit v3: the title rule, and every endpoint against a real SQL Server
`TaskApi.slnx`, `global.json`	The solution, and the SDK and test runner it needs
`frontend/`	React app, its tests, nginx.conf and Dockerfile
`k8s/`, `docker-compose.yml`	Kubernetes manifests and local Compose
`walkthrough/`	This book and the script that builds it

This book is generated from the source by `walkthrough/build_walkthrough.py`. Every listing is the real file with its real line numbers, and the bold "Line n" notes refer to them.

Part 1

The API

Three C# files. The entity and its rules, the database context, and Program.cs, which is the whole web app: configuration, startup and every endpoint, in about ninety lines.

1.1 The project file

MSBuild api/TaskApi.csproj, lines 1–15

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <!-- Write packages.lock.json: the exact version of every package, direct and transitive. -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.15" />
  </ItemGroup>

</Project>
```

Line 1 The Web SDK brings in ASP.NET Core and its implicit usings (`WebApplication`, `Results` and so on).

Line 5 Nullable reference types: `string` can't be null, `string?` can, and the compiler warns where you mix them up.

Line 8 Write `packages.lock.json`: NuGet's lock file. Part 5 covers it.

Line 12 One package. It brings EF Core, the SQL Server provider and `Microsoft.Data.SqlClient` with it.

1.2 TaskItem.cs: the entity and its rules

C# api/TaskItem.cs, lines 1–28

```
using System.ComponentModel.DataAnnotations;

namespace TaskApi;

public class TaskItem
{
    public int Id { get; set; }

    [MaxLength(TaskRules.MaxTitleLength)]
    public required string Title { get; set; }

    public bool Done { get; set; }

    public DateTimeOffset CreatedAt { get; set; } = DateTimeOffset.UtcNow;
}

public record CreateTaskRequest(string? Title);

public record UpdateTaskRequest(string? Title, bool? Done);

public static class TaskRules
{
    public const int MaxTitleLength = 200;

    /// <summary>A title needs something other than spaces in it, and at most 200 characters.</summary>
    public static bool IsValidTitle(string? title) =>
```

```
27          !string.IsNullOrWhiteSpace(title) && title.Length <= MaxTitleLength;
28  }
```

Line 7 EF Core's convention: a property called `Id` is the primary key, and an `int` key becomes an `IDENTITY` column.

Line 9 The column becomes `nvarchar(200)`. Without it, `nvarchar(max)`, which can't be indexed. The same constant drives the validation.

Line 10 `required`: code that creates a `TaskItem` must set `Title`, or it doesn't compile. Non-nullable, so the column is NOT NULL.

Line 14 `DateTimeOffset` keeps the offset, so the instant is unambiguous; SQL Server stores it as `datetimeoffset`.

Line 17 Request bodies as **records**: one line each. `Title` is nullable because a client can leave it out.

Line 19 `bool?` tells "not sent" (null) apart from `false`.

Line 26 Pulled out of Program.cs in 1.1.0 so it could be unit-tested without starting the app.

1.3 AppDbContext.cs

C# api/AppDbContext.cs, lines 1–8

```
1  using Microsoft.EntityFrameworkCore;
2
3  namespace TaskApi;
4
5  public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
6  {
7      public DbSet<TaskItem> Tasks => Set<TaskItem>();
8  }
```

Line 5 A **primary constructor** (C# 12): the options go straight to the base class. Which database and connection string is decided by whoever creates it, in Program.cs.

Line 7 A `DbSet` is a queryable table. LINQ on it (`db.Tasks.OrderBy(t => t.Id)`) is translated to SQL.

1.4 Program.cs: setup

C# api/Program.cs, lines 5–18

```
5  var builder = WebApplication.CreateBuilder(args);
6
7  // Connection string comes from ConnectionStrings__Default (set by docker-compose / Kubernetes),
8  // falling back to appsettings.json for running outside containers.
9  builder.Services.AddDbContext<AppDbContext>(options =>
10     options.UseSqlServer(
11         builder.Configuration.GetConnectionString("Default"),
12         sql => sql.EnableRetryOnFailure()));
13
14 var app = builder.Build();
15
16 await EnsureDatabaseAsync(app);
17
18 var api = app.MapGroup("/api");
```

Line 9 Registers the context with **dependency injection**, one instance per request. Handlers ask for it by type.

Line 11 Reads `ConnectionStrings:Default`. appsettings.json has a localhost default; the environment variable `ConnectionStrings__Default` (double underscore for the colon) overrides it in Compose and Kubernetes.

Line 12 Retry transient SQL Server errors (a dropped connection, a failover) automatically, with back-off.

Line 16 Create the database before accepting requests.

Line 18 A **route group**: every endpoint below is under `/api`.

1.5 Program.cs: endpoints

C# api/Program.cs, lines 20–30

```
20  // Liveness: the process is up. Kubernetes restarts the pod if this fails.
21  api.MapGet("/health/live", () => Results.Ok(new { status = "ok" }));
22
23  // Readiness: the database is reachable. Kubernetes stops sending traffic if this fails.
24  api.MapGet("/health/ready", async (AppDbContext db) =>
25      await db.Database.CanConnectAsync()
26          ? Results.Ok(new { status = "ok" })
27          : Results.Json(new { status = "database unavailable" }, statusCode: 503));
28
29  // Which pod answered. Inside Kubernetes the hostname is the pod name.
30  api.MapGet("/info", () => Results.Ok(new { hostname = Dns.GetHostName() }));
```

Line 21 Liveness doesn't touch the database, so a database outage doesn't make Kubernetes restart the API.

Line 25 Readiness does: 503 while the database is unreachable, and Kubernetes sends traffic elsewhere.

C# api/Program.cs, lines 32–33

```
32  api.MapGet("/tasks", async (AppDbContext db) =>
33      await db.Tasks.OrderBy(t => t.Id).ToListAsync());
```

A minimal API handler is a lambda. ASP.NET Core looks at its parameters: `AppDbContext` is a registered service, so it comes from dependency injection. The returned list is serialized to JSON, in camelCase.

C# api/Program.cs, lines 35–38

```
35  api.MapPost("/tasks", async (CreateTaskRequest request, AppDbContext db) =>
36  {
37      if (!TaskRules.IsValidTitle(request.Title))
38          return Results.BadRequest(new { error = "title is required (1-200 characters)" });
```

Line 35 A complex type with no other source binds from the JSON body. Malformed JSON never reaches the handler: the framework answers 400.

Line 40 The `!` is safe because `IsValidTitle` just returned true; the compiler can't see across that call.

Line 42 EF Core sends the INSERT and reads back the new `IDENTITY` value into `task.Id`.

Line 43 201 with a `Location` header pointing at the new task. A test checks the header.

C# api/Program.cs, lines 46–49

```
46  api.MapPatch("/tasks/{id:int}", async (int id, UpdateTaskRequest request, AppDbContext db) =>
47  {
48      if (request.Title is not null && !TaskRules.IsValidTitle(request.Title))
49          return Results.BadRequest(new { error = "title must be 1-200 characters" });
```

Line 46 A **route constraint**: `{id:int}` only matches integers, so `/tasks/abc` is a 404 before the handler runs.

Line 51 Load by key. EF Core now **tracks** the object...

Line 55 ...so changing its properties is enough...

Line 42 ...and this writes an UPDATE of just the changed columns.

C# api/Program.cs, lines 61–64

```
61  api.MapDelete("/tasks/{id:int}", async (int id, AppDbContext db) =>
62  {
63      var deleted = await db.Tasks.Where(t => t.Id == id).ExecuteDeleteAsync();
64      return deleted > 0 ? Results.NoContent() : Results.NotFound(new { error = "task not found" }
          );
```

`ExecuteDeleteAsync` (EF Core 7+) sends one `DELETE ... WHERE Id = @id` and returns the number of rows deleted, without loading the task first. 0 rows means 404.

1.6 Program.cs: starting up

C# api/Program.cs, lines 69–89

```
69  // Creates the database and table on first start. SQL Server takes 10-30 seconds to
70  // boot, so keep retrying rather than crashing while it comes up.
71  static async Task EnsureDatabaseAsync(WebApplication app, int attempts = 30)
72  {
73      for (var i = 1; ; i++)
74      {
75          try
76          {
77              using var scope = app.Services.CreateScope();
78              var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
79              await db.Database.EnsureCreatedAsync();
80              return;
81          }
82          catch (Exception ex) when (i < attempts)
83          {
84              app.Logger.LogWarning("Database not ready (attempt {Attempt}/{Max}): {Message}",
85                  i, attempts, ex.Message);
86              await Task.Delay(TimeSpan.FromSeconds(3));
87          }
88      }
89  }
```

Line 77 The context is scoped (one per request). Outside a request you make a scope to get one.

Line 79 Creates the database and tables from the model if they don't exist. Fine for a demo; a real app would use migrations, which can change an existing schema.

Line 82 An **exception filter**: on the last attempt the filter is false, the exception isn't caught, and startup fails loudly.

C# api/Program.cs, lines 91–92

```
// Lets the tests start this app in memory with WebApplicationFactory<Program>.
public partial class Program { }
```

Top-level statements compile into an internal `Program` class. Declaring it public lets the test project write `WebApplicationFactory<Program>`.

1.7 Interview questions

What does EF Core change tracking do in the PATCH handler?

FindAsync returns a tracked entity; EF Core remembers its original values, so SaveChangesAsync can generate an UPDATE for just the properties that changed.

EnsureCreated or migrations?

EnsureCreated builds the schema only when the database doesn't exist and can't change it later. Migrations record each schema change and apply it to existing databases; that's what production needs.

Where does a minimal API handler's argument come from?

Route values and query strings for simple types, the JSON body for one complex type, and DI for registered services. Attributes like [FromQuery] override the inference.

Why bool? for Done?

So the API can tell "don't change it" (absent, null) from "set it to false".

Part 2

The front end

One React component, a typed API client, and nginx in front. The page never knows which backend it's talking to: every call goes to /api, and whoever answers there is the API. That's why this front end is shared, file for file, by the Node and the .NET versions of the tracker.

2.1 Where the page comes from

HTML frontend/index.html, lines 1–12

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Task Tracker</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

TSX frontend/src/main.tsx, lines 1–10

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./styles.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Line 6 React takes over the empty `<div id="root">`. The `!` tells TypeScript "this isn't null"; index.html guarantees it.

Line 7 In development only, React runs effects twice to flush out code that isn't safe to re-run. It's why the initial load in App.tsx must be harmless to repeat (it is: it just fetches).

TypeScript frontend/src/config.ts, lines 1–2

```ts
// Shown in the header so it's obvious which backend you're talking to.
export const STACK_LABEL = "React · ASP.NET Core · SQL Server";
```

The only file that differs between the two projects. This one says `React · ASP.NET Core · SQL Server`.

2.2 The API client

TypeScript frontend/src/api.ts, lines 1–6

```ts
export interface Task {
  id: number;
  title: string;
  done: boolean;
  createdAt: string;
}
```

`createdAt` is a `string` here but a date on the server: JSON has no date type, so it arrives as ISO text. The type says what's really on the wire.

TypeScript frontend/src/api.ts, lines 8–18

```typescript
async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`/api${path}`, {
    headers: { "Content-Type": "application/json" },
    ...init,
  });
  if (!res.ok) {
    const body = await res.json().catch(() => null);
    throw new Error(body?.error ?? `Request failed (${res.status})`);
  }
  return res.status === 204 ? (undefined as T) : res.json();
}
```

Line 9 A relative URL: the same origin that served the page. In development Vite forwards `/api`; in Docker and Kubernetes, nginx does.

Line 11 Spread after the default headers, so a caller could override them; method and body come from here.

Line 13 `fetch` only rejects on network failure. A 400 or 500 is a successful fetch, so the status has to be checked.

Line 14 Use the API's own `{ error }` message if there is one. A proxy's 502 page isn't JSON, so parsing can fail; then fall back to the status.

Line 17 204 No Content has no body, and calling `.json()` on it would throw.

TypeScript frontend/src/api.ts, lines 20–28

```typescript
export const api = {
  list: () => request<Task[]>("/tasks"),
  create: (title: string) =>
    request<Task>("/tasks", { method: "POST", body: JSON.stringify({ title }) }),
  update: (id: number, changes: Partial<Pick<Task, "title" | "done">>) =>
    request<Task>(`/tasks/${id}`, { method: "PATCH", body: JSON.stringify(changes) }),
  remove: (id: number) => request<void>(`/tasks/${id}`, { method: "DELETE" }),
  info: () => request<{ hostname: string }>("/info"),
};
```

`Partial<Pick<Task, "title" | "done">>` builds the update type from `Task`: pick two fields, make both optional. Rename `done` in `Task` and this follows; you can't send a field the API doesn't take.

2.3 The component

TSX frontend/src/App.tsx, lines 5–10

```tsx
export default function App() {
  const [tasks, setTasks] = useState<Task[]>([]);
  const [title, setTitle] = useState("");
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [servedBy, setServedBy] = useState<string | null>(null);
```

Five pieces of state. Each `useState` returns the current value and a setter; calling the setter re-renders the component with the new value.

TSX frontend/src/App.tsx, lines 12–29

```tsx
async function run(action: () => Promise<void>) {
  setError(null);
  try {
    await action();
  } catch (e) {
    setError(e instanceof Error ? e.message : String(e));
  }
}

async function refresh() {
  setTasks(await api.list());
  // Which API pod answered: handy for seeing Kubernetes load-balance across replicas.
  setServedBy((await api.info()).hostname);
}

useEffect(() => {
  run(refresh).finally(() => setLoading(false));
}, []);
```

Line 12 Every user action goes through `run`: clear the old error, try, and show any failure. Error handling is written once.

Line 24 With two API replicas behind a Kubernetes Service, refreshing shows the pod name change: load balancing you can see.

Line 29 An empty dependency list: run once, when the component first appears. `finally` clears the loading state whether the load worked or not.

TSX frontend/src/App.tsx, lines 31–40

```tsx
function add(e: FormEvent) {
  e.preventDefault();
  const trimmed = title.trim();
  if (!trimmed) return;
  run(async () => {
    await api.create(trimmed);
    setTitle("");
    await refresh();
  });
}
```

`e.preventDefault()` stops the browser's own form submit, which would reload the page. After a change, the page re-reads the list from the API rather than editing its own copy: one extra request, but what you see is what the database has.

TSX frontend/src/App.tsx, lines 64–90

```tsx
      {loading ? (
        <p className="muted">Loading…</p>
      ) : tasks.length === 0 ? (
        <p className="muted">No tasks yet. Add one above.</p>
      ) : (
        <ul>
          {tasks.map((t) => (
            <li key={t.id} className={t.done ? "done" : ""}>
              <label>
                <input
                  type="checkbox"
                  checked={t.done}
                  onChange={() => run(async () => { await api.update(t.id, { done: !t.done }); a
wait refresh(); })}
```

```
                />
                <span>{t.title}</span>
              </label>
              <button
                className="delete"
                aria-label={`Delete ${t.title}`}
                onClick={() => run(async () => { await api.remove(t.id); await refresh(); })}
              >
                ×
              </button>
            </li>
          ))}
        </ul>
      )}
```

Line 71 key lets React match list items between renders. The database id is stable; an array index would shift when a task is deleted.

Line 82 The button shows only ×; the label tells a screen reader which task it deletes.

2.4 Development and production

TypeScript frontend/vite.config.ts, lines 1–11

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// In local dev (npm run dev), /api calls are forwarded to an API running on :8080.
// In Docker and Kubernetes, nginx does this forwarding instead (see nginx.conf).
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: { "/api": "http://localhost:8080" },
  },
});
```

nginx frontend/nginx.conf, lines 1–17

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  # Forward API calls to the "api" service. Docker Compose and Kubernetes
  # both resolve that name to the API container(s) via their internal DNS.
  location /api/ {
    proxy_pass http://api:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }

  location / {
    try_files $uri /index.html;
  }
}
```

Line 9 "api" is a DNS name: the compose service in Docker, the Service in Kubernetes. Neither the page nor nginx needs an IP address.

Line 15 A file if it exists, otherwise the app. Standard for single-page apps, so a deep link doesn't 404.

Dockerfile frontend/Dockerfile, lines 1–14

```dockerfile
# ---- build stage: compile the React app to static files ----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
# npm ci installs exactly what package-lock.json records.
RUN npm ci
COPY . .
RUN npm run build

# ---- runtime stage: tiny nginx image serving the static files ----
FROM nginx:1.31-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

A **multi-stage build**. The first stage has Node, every dev dependency and the source; it produces a folder of static files. The final image is nginx plus that folder: no Node, no `node_modules`, a few tens of megabytes. `npm ci` installs exactly what `package-lock.json` records.

Under Docker Compose the page is on `http://localhost:8081`.

2.5 Testing the client without a browser

TypeScript frontend/tests/api.test.ts, lines 10–17

```typescript
/** Makes every fetch return this status and body, and remember what it was asked. */
function respond(status: number, body?: unknown) {
  calls = [];
  globalThis.fetch = (async (url: string, init?: RequestInit) => {
    calls.push({ url, init });
    return new Response(body === undefined ? null : JSON.stringify(body), { status });
  }) as typeof fetch;
}
```

The test replaces the global `fetch` with a fake that records the request and returns a canned `Response` (Node 18+ has the same `Response` class as the browser). `afterEach` puts the real one back.

TypeScript frontend/tests/api.test.ts, lines 54–55

```typescript
test("an error response throws the API's own message", async () => {
  respond(400, { error: "title is required (1-200 characters)" });
```

These check the edges of the client: the URL and method of each call, the JSON body, 204 handling, and both error paths. Run them with `npm test` in `frontend/`.

2.6 Interview questions

Why does fetch not throw on a 404?

It only rejects when there's no response at all (network error, CORS). Any HTTP response, including errors, resolves; you check `res.ok` or the status yourself.

Why proxy /api instead of calling the API's URL from the browser?

Same origin: no CORS, no API address baked into the build, and the same image works in Compose, Kubernetes or anywhere the name "api" resolves.

Why not update the task list locally after a change?

It's simpler and always matches the database. For a busier app you'd update optimistically and roll back on error.

Part 3

Containers and Kubernetes

The same shape as the Node version: three images, Compose locally, Kubernetes manifests applied with kustomize. SQL Server brings a few needs of its own: memory, a password policy, and a slow start.

3.1 The API image

Dockerfile api/Dockerfile, lines 1–17

```dockerfile
# ---- build stage: restore and publish with the .NET SDK ----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY TaskApi.csproj packages.lock.json ./
# --locked-mode restores exactly what packages.lock.json records, and fails if the project disag
      rees.
RUN dotnet restore --locked-mode
COPY . .
RUN dotnet publish -c Release -o /app --no-restore

# ---- runtime stage: ASP.NET runtime only (no SDK) ----
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app .
# The official image listens on 8080 and provides a non-root user.
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "TaskApi.dll"]
```

Line 4 The project and lock file first, so the restore layer is cached until dependencies change.

Line 6 Restore exactly what the lock records, or fail.

Line 11 The runtime image: no SDK, no compiler, much smaller.

Line 15 Microsoft's .NET images define a non-root user and set the port to 8080, which a non-root process is allowed to bind.

3.2 Docker Compose

YAML docker-compose.yml, lines 1–38

```yaml
# Runs the whole app locally with Docker: http://localhost:8081
#   docker compose up --build
name: tasks-dotnet

services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_SA_PASSWORD: "Tasks-dev-Passw0rd"
    volumes:
      - db-data:/var/opt/mssql
    healthcheck:
      test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P \"$$MSSQL_SA_PASS
      WORD\" -C -Q 'SELECT 1' || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 20s

  api:
    build: ./api
    image: tasks-dotnet-api:1.0
    environment:
      ConnectionStrings__Default: "Server=db,1433;Database=TaskDb;User Id=sa;Password=Tasks-dev-
      Passw0rd;TrustServerCertificate=True"
    depends_on:
      db:
```

```
27          condition: service_healthy
28
29    web:
30      build: ./frontend
31      image: tasks-dotnet-web:1.0
32      ports:
33        - "8081:80"
34      depends_on:
35        - api
36
37  volumes:
38    db-data:
```

Line 9 SQL Server won't start without accepting the licence. The Developer edition is the default.

Line 10 It must meet SQL Server's complexity rules, or the container exits.

Line 18 SQL Server takes a while to boot; failed checks in the first 20 seconds don't count.

Line 24 Overrides appsettings.json. `Server=db`: the Compose service name.

Line 33 8081, so it can run beside the Node version on 8080.

3.3 Kubernetes

YAML k8s/sqlserver.yaml, lines 15–65

```
15  kind: StatefulSet
16  metadata:
17    name: sqlserver
18  spec:
19    serviceName: sqlserver
20    replicas: 1
21    selector:
22      matchLabels:
23        app: sqlserver
24    template:
25      metadata:
26        labels:
27          app: sqlserver
28      spec:
29        securityContext:
30          # SQL Server runs as user 10001; this lets it write to the volume.
31          fsGroup: 10001
32        containers:
33          - name: sqlserver
34            image: mcr.microsoft.com/mssql/server:2022-latest
35            ports:
36              - containerPort: 1433
37            env:
38              - name: ACCEPT_EULA
39                value: "Y"
40              - name: MSSQL_SA_PASSWORD
41                valueFrom: { secretKeyRef: { name: db-credentials, key: MSSQL_SA_PASSWORD } }
42            volumeMounts:
43              - name: data
44                mountPath: /var/opt/mssql
45            readinessProbe:
46              exec:
47                command:
48                  - sh
49                  - -c
50                  - /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C -Q
          "SELECT 1"
51              initialDelaySeconds: 20
```

```
            periodSeconds: 10
            timeoutSeconds: 5
          resources:
            # SQL Server needs at least 2 GB of memory to start.
            requests: { cpu: 250m, memory: 2Gi }
            limits: { memory: 3Gi }
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 2Gi
```

Line 31 SQL Server's container runs as user 10001. `fsGroup` makes the volume writable by that group; without it, SQL Server can't create its files.

Line 50 Ready when it answers a query. `-C` trusts the server's self-signed certificate.

Line 56 SQL Server needs 2 GB to start. Docker Desktop's Kubernetes needs that much memory available.

YAML k8s/api.yaml, lines 26–50

```
      containers:
        - name: api
          image: tasks-dotnet-api:1.0
          # Uses the image you built locally; Docker Desktop's cluster shares Docker's images.
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          env:
            - name: ConnectionStrings__Default
              valueFrom: { secretKeyRef: { name: db-credentials, key: ConnectionStrings__Default
       } }
          # The API waits for the database before it starts listening. The startup probe
          # gives it up to 150s to do that before liveness checks take over.
          startupProbe:
            httpGet: { path: /api/health/live, port: 8080 }
            periodSeconds: 5
            failureThreshold: 30
          livenessProbe:
            httpGet: { path: /api/health/live, port: 8080 }
            periodSeconds: 10
          readinessProbe:
            httpGet: { path: /api/health/ready, port: 8080 }
            periodSeconds: 5
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { memory: 512Mi }
```

Line 34 One setting carries the whole connection string, read from the Secret.

Line 38 Up to 150 seconds to start while `EnsureDatabaseAsync` waits for SQL Server; liveness takes over afterwards.

YAML k8s/db-secret.yaml, lines 1–11

```
# Demo credentials only. In a real cluster these would come from a secret
# manager (or be created with `kubectl create secret`), not live in git,
# and the app would use its own SQL login instead of sa.
apiVersion: v1
kind: Secret
metadata:
```

```
  name: db-credentials
type: Opaque
stringData:
  MSSQL_SA_PASSWORD: "Tasks-k8s-Passw0rd"
  ConnectionStrings__Default: "Server=sqlserver,1433;Database=TaskDb;User Id=sa;Password=Tasks-k
      8s-Passw0rd;TrustServerCertificate=True"
```

The same Kubernetes ideas as the Node book: headless Service plus StatefulSet for the database, a Deployment with two replicas for the API, a LoadBalancer Service for the page, and a namespace (`tasks-dotnet`) that holds it all.

3.4 Interview questions

Why does the API retry on startup when Compose already waits for the database?

Kubernetes doesn't order startup between pods, and a database can restart while the API is running. Retries and probes handle both.

What would change for production?

A managed database or a properly sized StatefulSet with backups, a dedicated SQL login instead of sa, secrets from a secret manager, EF Core migrations, and images from a registry with pinned tags.

Part 4

Tests

xUnit v3. Nine fast tests for the title rule, and twelve that run the real API end to end, through HTTP, EF Core and a real SQL Server database created for each test and dropped after.

4.1 The test project

MSBuild tests/TaskApi.Tests/TaskApi.Tests.csproj, lines 1–28

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <!-- xUnit v3 test projects are small console apps. -->
    <OutputType>Exe</OutputType>
    <IsPackable>false</IsPackable>
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.15" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.12.0" />
    <PackageReference Include="xunit.v3" Version="4.0.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="4.0.1" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\..\api\TaskApi.csproj" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

</Project>
```

Line 8 xUnit v3 test projects are programs: each one can run on its own.

Line 14 Provides `WebApplicationFactory`.

Line 21 The tests reference the real app.

Line 25 A global `using Xunit;` for every file.

JSON global.json, lines 1–9

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  },
  "test": {
    "runner": "Microsoft.Testing.Platform"
  }
}
```

Line 4 Any .NET 10 SDK from 10.0.100 up.

Line 7 Current xUnit v3 runs on Microsoft.Testing.Platform. On the .NET 10 SDK, `dotnet test` must be told so, and then takes `--solution` or `--project`. Without it, the first CI run failed with "Testing with VSTest target is no longer supported".

4.2 Unit tests: the title rule

C# tests/TaskApi.Tests/TaskRulesTests.cs, lines 5–29

```
public class TaskRulesTests
{
    [Theory]
    [InlineData("a")]
    [InlineData("Buy milk")]
    [InlineData("  padded  ")]
    public void Accepts_titles_with_text(string title) =>
        Assert.True(TaskRules.IsValidTitle(title));

    [Fact]
    public void Accepts_exactly_the_maximum_length() =>
        Assert.True(TaskRules.IsValidTitle(new string('x', TaskRules.MaxTitleLength)));

    [Theory]
    [InlineData(null)]
    [InlineData("")]
    [InlineData("   ")]
    [InlineData("\t\n")]
    public void Rejects_missing_or_blank_titles(string? title) =>
        Assert.False(TaskRules.IsValidTitle(title));

    [Fact]
    public void Rejects_one_character_over_the_maximum() =>
        Assert.False(TaskRules.IsValidTitle(new string('x', TaskRules.MaxTitleLength + 1)));
}
```

A `[Theory]` runs once per `[InlineData]` row. The boundaries are tested exactly: 200 characters passes, 201 doesn't. These run anywhere, with nothing installed.

4.3 A real app with a throwaway database

C# tests/TaskApi.Tests/SqlServerApiFactory.cs, lines 7–49

```
/// <summary>
/// Starts the real API in memory (WebApplicationFactory), pointed at a brand-new SQL Server
/// database that's dropped afterwards. The server comes from the TEST_SQLSERVER environment
/// variable, a connection string without a database name; CI starts one in a container.
/// Without it, the HTTP tests are skipped rather than failing.
/// </summary>
public sealed class SqlServerApiFactory : WebApplicationFactory<Program>, IAsyncLifetime
{
    public static readonly string? SqlServer = Environment.GetEnvironmentVariable("TEST_SQLSERVE
      R");

    private readonly string _database = $"TaskDb_Test_{Guid.NewGuid():N}";

    private string ConnectionString =>
        new SqlConnectionStringBuilder(SqlServer) { InitialCatalog = _database }.ConnectionStrin
      g;

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        // Program reads ConnectionStrings:Default; this setting wins over appsettings.json.
        builder.UseSetting("ConnectionStrings:Default", ConnectionString);
    }

    public ValueTask InitializeAsync() => ValueTask.CompletedTask;

    public override async ValueTask DisposeAsync()
    {
        await base.DisposeAsync();
```

```
33          if (SqlServer is null) return;
34
35          // Drop the throwaway database. SINGLE_USER closes pooled connections still holding it o
            pen.
36          SqlConnection.ClearAllPools();
37          await using var connection = new SqlConnection(SqlServer);
38          await connection.OpenAsync();
39          await using var command = connection.CreateCommand();
40          command.CommandText = $"""
41              IF DB_ID('{_database}') IS NOT NULL
42              BEGIN
43                  ALTER DATABASE [{_database}] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
44                  DROP DATABASE [{_database}];
45              END
46              """;
47          await command.ExecuteNonQueryAsync();
48      }
49  }
```

Line 15 A connection string to a server, without a database. (Named `SqlServer` because the base class already has a `Server`; reusing the name was a compile warning, and CI fails on warnings.)

Line 17 A new, uniquely named database for every factory, so tests can't see each other's data.

Line 25 Overrides the app's connection string. When the app starts, `EnsureDatabaseAsync` creates this database.

Line 36 The app's pooled connections keep the database in use, and in-use databases can't be dropped...

Line 43 ...so also disconnect anything left, then drop it.

C# tests/TaskApi.Tests/TasksApiTests.cs, lines 6–34

```
6   /// <summary>
7   /// The API over HTTP, end to end: routing, validation, EF Core and a real SQL Server database.
8   /// Each test gets its own empty database (a new factory per test), so they can't affect each ot
        her.
9   /// </summary>
10  public sealed class TasksApiTests : IAsyncLifetime
11  {
12      private SqlServerApiFactory _factory = null!;
13      private HttpClient _client = null!;
14
15      private record TaskDto(int Id, string Title, bool Done, DateTimeOffset CreatedAt);
16
17      private record ErrorDto(string Error);
18
19      public ValueTask InitializeAsync()
20      {
21          Assert.SkipWhen(SqlServerApiFactory.SqlServer is null,
22              "Set TEST_SQLSERVER to a SQL Server connection string (no database name) to run the
        HTTP tests.");
23          _factory = new SqlServerApiFactory();
24          _client = _factory.CreateClient();
25          return ValueTask.CompletedTask;
26      }
27
28      public async ValueTask DisposeAsync()
29      {
30          _client?.Dispose();
31          if (_factory is not null) await _factory.DisposeAsync();
32      }
33
34      private static CancellationToken Ct => TestContext.Current.CancellationToken;
```

Line 21 xUnit v3 can skip at run time. Without `TEST_SQLSERVER` these tests are reported as skipped, with this message, not failed.

Line 23 xUnit creates a new instance of the class for every test, so every test gets its own app and database.

Line 34 xUnit's analyzer `xUnit1051` asks async calls to take the test's token, so a cancelled run stops promptly. With `-warnaserror` in CI, leaving it out fails the build.

C# tests/TaskApi.Tests/TasksApiTests.cs, lines 51–60

```
51      public async Task Post_creates_a_trimmed_task_with_a_location_header()
52      {
53          var response = await _client.PostAsJsonAsync("/api/tasks", new { title = "  Buy milk  "
          }, Ct);
54
55          Assert.Equal(HttpStatusCode.Created, response.StatusCode);
56          var task = await response.Content.ReadFromJsonAsync<TaskDto>(Ct);
57          Assert.Equal("Buy milk", task!.Title);
58          Assert.False(task.Done);
59          Assert.Equal($"/api/tasks/{task.Id}", response.Headers.Location?.ToString());
60      }
```

C# tests/TaskApi.Tests/TasksApiTests.cs, lines 107–116

```
107     public async Task Patch_rejects_a_blank_title_and_leaves_the_task_alone()
108     {
109         var task = await CreateAsync("Keep me");
110
111         var response = await _client.PatchAsJsonAsync($"/api/tasks/{task.Id}", new { title = " "
            }, Ct);
112
113         Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
114         var tasks = await _client.GetFromJsonAsync<List<TaskDto>>("/api/tasks", Ct);
115         Assert.Equal("Keep me", Assert.Single(tasks!).Title);
116     }
```

Not only the 400, but that the stored task didn't change. `Assert.Single` also checks there's exactly one task.

C# tests/TaskApi.Tests/TasksApiTests.cs, lines 129–134

```
129     public async Task A_non_numeric_id_does_not_match_the_route()
130     {
131         // {id:int} in the route means "abc" never reaches the handler.
132         var response = await _client.DeleteAsync("/api/tasks/abc", Ct);
133         Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
134     }
```

4.4 Running them

Command	Runs
`dotnet test --solution TaskApi.slnx`	All 21; the 12 HTTP tests skip without a server
`docker compose up -d db` (with `ports: ["1433:1433"]` added under db)	A SQL Server to test against
`$env:TEST_SQLSERVER = "Server=localhost,1433;User Id=sa;Password=...;TrustServerCertificate=True"`	PowerShell; then `dotnet test` runs all 21
`npm test` in frontend/	The 6 client tests

4.5 Interview questions

What does WebApplicationFactory give you?

The real app, started in memory with its real configuration and DI, plus an HttpClient wired to it with no network. You can override settings and services per test.

Why a real SQL Server instead of EF Core's in-memory provider?

The in-memory provider isn't a relational database: no SQL, no constraints, different behaviour for strings and transactions. Tests against it can pass while the real query fails.

Why skip rather than fail without a database?

So `dotnet test` works on any machine. CI runs them both ways, so a test that wrongly fails without a database, and one that silently skips with it, are both caught.

Part 5

Keeping it working

Lock files for NuGet and npm, versions that mean something, and CI that tests against a real SQL Server on every push. Plus one real vulnerability the checks found.

5.1 NuGet lock files

`packages.lock.json` sits beside each project and records every package, direct and transitive, with its exact version and content hash. `dotnet restore --locked-mode` installs exactly that and fails if the project asks for something else: NuGet's `npm ci`.

It has one gap: a project with no lock file at all restores without complaint. So CI checks first that every project has one:

YAML ../../.github/workflows/fullstack-dotnet-sqlserver.yml, lines 39–48

```yaml
      # --locked-mode only enforces a lock file that exists, so check every project has one.
      - name: Every project has a committed lock file
        shell: bash
        run: |
          missing=0
          for proj in $(git ls-files '*.csproj'); do
            lock="$(dirname "$proj")/packages.lock.json"
            git ls-files --error-unmatch "$lock" > /dev/null 2>&1 || { echo "::error file=$proj:
      :no committed $lock"; missing=1; }
          done
          exit $missing
```

5.2 A vulnerable package nobody chose

Version 1.0.0 referenced EF Core 10.0.0. Restoring it warned `NU1903`: `System.Security.Cryptography.Pkcs` had a known high-severity vulnerability. The project never mentions Pkcs; it arrives through EF Core's SQL Server provider and `Microsoft.Data.SqlClient`: a **transitive dependency**.

The fix was EF Core 10.0.15, a **patch** release whose dependencies include a fixed Pkcs. Under Semantic Versioning a patch changes nothing you rely on, so the app needed no other change, and the changelog records it under 1.1.0. To keep it from happening quietly again:

YAML ../../.github/workflows/fullstack-dotnet-sqlserver.yml, lines 59–62

```yaml
      - name: Vulnerable packages
        run: |
          dotnet list TaskApi.slnx package --vulnerable --include-transitive | tee vulnerable.tx
      t
          ! grep -q -E "High|Critical" vulnerable.txt
```

If no patched version of the direct dependency exists, the other fix is to reference the patched transitive package directly. NuGet picks the lowest version that satisfies every request, so the direct reference wins. That's what the Genetec Access Event Log does.

5.3 Semantic Versioning

`CHANGELOG.md` defines the contract: the REST API (routes, request and response fields) and how the app is run (images, environment variables). Breaking either is a major version. 1.1.0 added tests, `TaskRules` and `public partial class Program`, and patched EF Core, with no change in behaviour: a minor version.

5.4 The workflow

YAML ../../.github/workflows/fullstack-dotnet-sqlserver.yml, lines 18–62

```yaml
18  api:
19    runs-on: ubuntu-latest
20    defaults:
21      run:
22        working-directory: Full Stack/dotnet-sqlserver
23    # A throwaway SQL Server. Each HTTP test creates its own database on it and drops it after.
24    services:
25      sqlserver:
26        image: mcr.microsoft.com/mssql/server:2022-latest
27        env:
28          ACCEPT_EULA: "Y"
29          MSSQL_SA_PASSWORD: Tasks-ci-Passw0rd
30        ports: ["1433:1433"]
31        options: >-
32          --health-cmd "/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P Tasks-ci-Passw0rd -C
     -Q 'SELECT 1' || exit 1"
33          --health-interval 10s --health-timeout 5s --health-retries 12 --health-start-period 20
    s
34    steps:
35      - uses: actions/checkout@v7
36      - uses: actions/setup-dotnet@v6
37        with:
38          global-json-file: Full Stack/dotnet-sqlserver/global.json
39      # --locked-mode only enforces a lock file that exists, so check every project has one.
40      - name: Every project has a committed lock file
41        shell: bash
42        run: |
43          missing=0
44          for proj in $(git ls-files '*.csproj'); do
45            lock="$(dirname "$proj")/packages.lock.json"
46            git ls-files --error-unmatch "$lock" > /dev/null 2>&1 || { echo "::error file=$proj:
    :no committed $lock"; missing=1; }
47          done
48          exit $missing
49      # --locked-mode: restore exactly what the packages.lock.json files record, or fail.
50      - run: dotnet restore TaskApi.slnx --locked-mode
51      - run: dotnet build TaskApi.slnx --no-restore -c Release -warnaserror
52      # Without a server the HTTP tests must skip, not fail: that's how they behave on a laptop.
53      - name: Tests without a database (HTTP tests skip)
54        run: dotnet test --solution TaskApi.slnx --no-build -c Release
55      - name: Tests against SQL Server
56        run: dotnet test --solution TaskApi.slnx --no-build -c Release
57        env:
58          TEST_SQLSERVER: Server=localhost,1433;User Id=sa;Password=Tasks-ci-Passw0rd;TrustServe
    rCertificate=True
59      - name: Vulnerable packages
60        run: |
61          dotnet list TaskApi.slnx package --vulnerable --include-transitive | tee vulnerable.tx
    t
62          ! grep -q -E "High|Critical" vulnerable.txt
```

Line 26 A real SQL Server as a **service container**, reachable on `localhost:1433` once its health check passes.

Line 38 CI installs the SDK `global.json` asks for.

Line 51 Warnings fail the build.

Line 53 Run once without `TEST_SQLSERVER`: the HTTP tests must skip, as on a laptop...

Line 55 ...and once with it: all 21 must pass.

YAML ../../.github/workflows/fullstack-dotnet-sqlserver.yml, lines 64–89

```yaml
  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/dotnet-sqlserver/frontend
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version-file: Full Stack/dotnet-sqlserver/frontend/.nvmrc
          cache: npm
          cache-dependency-path: Full Stack/dotnet-sqlserver/frontend/package-lock.json
      - run: npm ci
      - run: npm test
      - run: npm run build
      - run: npm audit --omit=dev --audit-level=high

  images:
    name: docker images
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: Full Stack/dotnet-sqlserver
    steps:
      - uses: actions/checkout@v7
      - run: docker compose build
```

The front end's tests, build and `npm audit`, and a `docker compose build` to catch a broken Dockerfile. All on every push to any branch and every pull request.

5.5 Dependabot

YAML ../../.github/dependabot.yml, lines 57–70

```yaml
  - package-ecosystem: nuget
    directory: "/Full Stack/dotnet-sqlserver"
    schedule:
      interval: weekly
      day: monday
    commit-message:
      prefix: "full stack"
    groups:
      minor-and-patch:
        update-types: [minor, patch]
    ignore:
      # EF Core and ASP.NET Core majors follow .NET: 11.x needs the project moved to net11.0.
      - dependency-name: "Microsoft.*"
        update-types: ["version-update:semver-major"]
```

Weekly NuGet updates for this project, minor and patch bundled into one pull request. Major versions of Microsoft's packages are skipped: EF Core and ASP.NET Core 11 need .NET 11, so they arrive with a deliberate move of the whole project, not as a package bump. The front end's npm packages and the Dockerfiles' base images (`dotnet/sdk:10.0`, `dotnet/aspnet:10.0`, `nginx`)

have entries of their own. Every pull request runs the workflow, against SQL Server.

A green build that wouldn't start

Dependabot's first run offered `dotnet/sdk:11.0` and `dotnet/aspnet:11.0` for the Dockerfile. CI built the image, passed, and they were merged. But the project still targets `net10.0`, and the .NET runtime only rolls forward to newer *minor* versions by default: a .NET 10 app on the .NET 11 runtime image stops at once with "You must install or update .NET to run this application". The image built fine; it just couldn't run.

Nothing in CI started the container, so nothing noticed. The images went back to 10.0, and Dependabot now skips majors of the `dotnet/*` images. A smoke test would have caught it: start the container in CI and request `/api/health/live` (exercise below).

5.6 Interview questions

What does packages.lock.json add when the .csproj has exact versions?

The transitive graph and content hashes. The project pins only what it references directly.

How do you find where a transitive package comes from?

`dotnet list package --include-transitive`, or `dotnet nuget why <project> <package>` on recent SDKs, which prints the dependency path.

CI built the Docker image and passed, but the container won't start. How?

Building only proves it compiles and packages. Here the runtime image was a newer .NET major than the app targets, which the runtime won't run by default. Start the container in CI and hit a health endpoint, and keep the SDK, runtime image and TargetFramework on the same major.

Why run the tests twice in CI?

Once to prove they skip cleanly without a database, once to prove they pass with one.

5.7 Exercise

Smoke-test the image

Add a CI step that runs the API's image and checks it answers.

Hint. `docker compose up -d --wait` with the database, then `curl -f localhost:8081/api/health/live`. Try it against `dotnet/aspnet:11.0` to watch it catch the problem in 5.5.